



Reading Between the Lines: Towards Reliable Black-box LLM Fingerprinting via Zeroth-order Gradient Estimation

Shuo Shao
Zhejiang University
Hangzhou, China
shaoshuo_ss@zju.edu.cn

Yiming Li*
Nanyang Technological University
Singapore, Singapore
liyiming.tech@gmail.com

Hongwei Yao
City University of Hong Kong
Hong Kong, China
yao.hongwei@cityu.edu.hk

Yifei Chen
Zhejiang University
Hangzhou, China
yifei.chen@zju.edu.cn

Yuchen Yang
Zhejiang University
Hangzhou, China
ychyang@zju.edu.cn

Zhan Qin
Zhejiang University
Hangzhou, China
qinzhan@zju.edu.cn

Abstract

The substantial investment required to develop Large Language Models (LLMs) makes them valuable intellectual property, raising significant concerns about copyright protection. LLM fingerprinting has emerged as a key technique to address this, which aims to verify a model's origin by extracting an intrinsic, unique signature (a "fingerprint") and comparing it to that of a source model to identify illicit copies. However, existing black-box fingerprinting methods often fail to generate distinctive LLM fingerprints. This ineffectiveness arises because black-box methods typically rely on model outputs, which lose critical information about the model's unique parameters due to the usage of non-linear functions. To address this, we first leverage Fisher Information Theory to formally demonstrate that the gradient of the model's input is a more informative feature for fingerprinting than the output. Based on this insight, we propose ZEROPRINT, a novel method that approximates these information-rich gradients in a black-box setting using zeroth-order estimation. ZEROPRINT overcomes the challenge of applying this to discrete text by simulating input perturbations via semantic-preserving word substitutions. This operation allows ZEROPRINT to estimate the model's Jacobian matrix as a unique fingerprint. Experiments on the standard benchmark show ZEROPRINT achieves a state-of-the-art effectiveness and robustness, significantly outperforming existing black-box methods.

CCS Concepts

• Security and privacy; • Computing methodologies → Natural language processing;

Keywords

Model Fingerprinting; AI Copyright Protection; Large Language Model; Trustworthy AI

*Corresponding Author.



This work is licensed under a Creative Commons Attribution 4.0 International License. *WWW '26, Dubai, United Arab Emirates*
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2307-0/2026/04
<https://doi.org/10.1145/3774904.3792196>

ACM Reference Format:

Shuo Shao, Yiming Li, Hongwei Yao, Yifei Chen, Yuchen Yang, and Zhan Qin. 2026. Reading Between the Lines: Towards Reliable Black-box LLM Fingerprinting via Zeroth-order Gradient Estimation. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3774904.3792196>

Resource Availability:

The source code of this paper has been made publicly available at <https://doi.org/10.5281/zenodo.18325480> and <https://github.com/shaoshuo-ss/ZeroPrint>.

1 Introduction

Large Language Models (LLMs) have become fundamental in revolutionizing the digital landscape [12, 28], powering a vast array of web applications from intelligent search engines [35, 48] and conversational agents [6, 26] to sophisticated content creation platforms [5]. However, the development of these state-of-the-art (SOTA) models demands substantial investment in computational resources, massive datasets, and extensive research expertise, rendering them highly valuable intellectual property for their developers [46, 52, 54]. Consequently, this value has given rise to significant security concerns regarding their copyright. Adversaries may engage in copyright infringement by using open-source models for commercial purposes without authorization [37], or even resort to model stealing [1, 51], where proprietary models may be illicitly replicated and deployed. Protecting the copyright of these models is therefore not just a legal formality but a critical necessity to promote sustainable innovation and maintain a trustworthy AI ecosystem [33, 50].

To address copyright concerns, LLM fingerprinting has emerged as a promising solution [11, 29, 52, 54]. This technique aims to determine whether a third-party suspicious model is derived from a copyrighted source model by extracting its intrinsic and unique features (*i.e.*, a fingerprint) in a non-invasive manner [2, 24, 57]. Copyright attribution is then performed by measuring the similarity between the fingerprints of the two models. Compared to invasive LLM watermarking techniques [20, 38, 49], which proactively embed a verifiable signal into a model's parameters or outputs before its release, fingerprinting offers significant advantages [37]. First, it does not require model fine-tuning, which means it has a smaller overhead and also does not degrade the model's performance. Second, it can be applied to protect models that have already been

deployed or released, a scenario where watermarking is not a viable option. Consequently, LLM fingerprinting has become a more practical and mainstream solution for copyright auditing.

Existing LLM fingerprinting techniques are primarily classified based on the level of access to the suspicious model, leading to two categories: white-box and black-box methods [37]. White-box fingerprinting assumes full access to a model's internal components, including its architecture and parameters. This allows fingerprints to be extracted directly from static model weights [52, 57], intermediate representations [53, 54], or gradients of weights [47]. Conversely, black-box fingerprinting operates under the constraint that the model developer can only interact with the model through the API and observe the corresponding outputs. This paradigm is further divided into two strategies. The first, *untargeted fingerprinting* [17, 29, 39], involves submitting a selected set of queries to different models and comparing their outputs, where high similarity can indicate copyright infringement. The second strategy, *targeted fingerprinting* [11, 44], generates a unique set of query-response pairs by optimizing them on a source model and then verifies if a suspicious model can consistently reproduce these specific outputs.

However, a significant performance gap persists between white-box and black-box fingerprinting methods. A recent study [37] reveals that white-box techniques, especially static methods that extract features directly from model parameters, achieve near-perfect auditing performance. This effectiveness likely stems from the immense scale of LLM parameters, where the distinct training paths of independently developed models cause their static weights to become inherently unique identifiers. The success of static fingerprinting thus suggests that *model parameters may be the most critical features for reliable auditing*. A key challenge for black-box fingerprinting is therefore how to extract features that retain as much information about these parameters as possible. Existing black-box methods primarily rely on comparing model outputs. However, the extensive use of non-linear functions (*e.g.*, GELU and SiLU [15]) throughout an LLM's architecture progressively compresses parameter information, leading to significant information loss in the final output. This raises a critical research question for the field:

Is there another feature, accessible under black-box conditions, that can preserve more information about model parameters than output?

To address this question, we first ground our investigation in a formal analysis of feature information content based on Fisher Information Theory [23]. Our theoretical analysis reveals a crucial insight: given the input X , the amount of information about the parameter W that can be obtained by observing the gradient $D = dY/dX$ is approximately always greater than that by observing the output Y . This finding suggests that the gradient is an inherently more distinctive feature for a reliable fingerprint. Based on this understanding, we propose ZEROPRINT, a novel LLM fingerprinting method that captures this information-rich gradient in a black-box setting. The core challenge is that gradients are not directly accessible in the black-box auditing scenario. To overcome this, we adapt the principles of zeroth-order optimization [4], a technique designed to approximate gradients using only input-output queries. However, classical zeroth-order methods require applying small, continuous perturbations to inputs, which is a process not directly applicable to the discrete and symbolic nature of text. ZEROPRINT tackles this gap by devising a method to simulate these

perturbations in the text domain by generating semantically preserved variations of base queries through word substitution. By submitting these base and perturbed queries to a model, we can observe the corresponding changes in its output embeddings. These input-output difference pairs then allow us to estimate the local Jacobian matrix of the model using a regression-based approach, which serves as its unique fingerprint. By comparing the Jacobian fingerprint of a suspicious model with that of a source model, we can reliably determine its provenance.

Our contributions can be summarized as follows.

- We provide a formal theoretical analysis grounded in Fisher Information Theory. Our analysis mathematically demonstrates that a model's gradient with respect to its input contains more information about the model's parameters than its output does, establishing gradients as a more robust feature for fingerprinting.
- We introduce ZEROPRINT, a novel black-box fingerprinting framework that estimates gradients as fingerprints without direct access to the model. ZEROPRINT overcomes the challenge of operating in a discrete text domain by simulating input perturbations through semantic-preserving word substitutions, enabling the approximation of the Jacobian matrix as a distinctive fingerprint.
- We conduct extensive experiments on LEAFBENCH, a representative benchmark for LLM copyright auditing. The results show that ZEROPRINT consistently outperforms existing SOTA black-box fingerprinting methods across various metrics, demonstrating its superior effectiveness and reliability.

2 Background

2.1 Large Language Models

Large Language Models (LLMs) have become a cornerstone of the modern web, fundamentally reshaping how users interact with online services and information [6, 22, 58]. An LLM is a sophisticated computational system, typically defined by its vast parameter scale [10, 12, 42]. The generation of text by these models follows an auto-regressive paradigm. This means that the model produces text sequentially, one token at a time, where each new token is conditionally dependent on the sequence of previously generated tokens. The probability of generating a sequence of tokens $Y = (y_1, y_2, \dots, y_T)$ is factorized into a product of conditional probabilities:

$$P(Y) = \prod_{t=1}^T P(y_t | y_1, y_2, \dots, y_{t-1}). \quad (1)$$

This underlying mechanism enables the diverse web applications, such as search engines [35], code generation [13, 25], and web agents [6, 26]. LLMs represent one of the most significant advancements in computing and have become immensely valuable digital assets. Consequently, protecting the copyright of these models within the web environment has emerged as a critical challenge.

2.2 LLM Fingerprinting

As LLMs are integrated into a growing number of applications, protecting the intellectual property of these models has become a critical task [19, 50]. LLM fingerprinting is a key technique that has emerged for this purpose, designed to verify a model's provenance by analyzing its inherent characteristics [37]. In this process, a copyright auditor can extract a unique signature (*i.e.*, a fingerprint) from a suspicious model and compare it against the fingerprint of

the source model to determine if it is an illicit copy or an unauthorized derivative. LLM fingerprinting provides a reliable mechanism for model owners to audit and protect their digital assets. Broadly, existing LLM fingerprinting can be categorized into two types: white-box and black-box LLM fingerprinting [37].

White-box LLM Fingerprinting. White-box fingerprinting methods operate under the assumption that the auditor has full access to a model’s internal components, including its architecture and parameters. These techniques are categorized based on their feature source into three different methods: static, forward-pass, and backward-pass methods [37]. Static fingerprinting directly analyzes a model’s learned parameters as the primary identifier [52, 57]. Forward-pass methods leverage the dynamic intermediate representations generated as the model processes inputs [53, 54], while backward-pass techniques use the gradients produced during back-propagation as a unique signature [47]. Although effective, the practical application of these methods is limited to open-source models where model parameters have already been accessible.

Black-box LLM Fingerprinting. In contrast, black-box fingerprinting methods are more broadly applicable, as they only require API-level access to query a model and observe its outputs. These approaches can be divided into two main categories: untargeted and targeted fingerprinting [37].

- **Untargeted Fingerprinting:** This is a straightforward and intuitive approach. The core idea is based on the observation that different LLMs often exhibit unique stylistic or behavioral “idiosyncrasies” in their generated outputs [39]. Untargeted methods first select a set of queries, then submit them to different models and analyze the characteristics of the responses to generate a fingerprint [9, 24, 29, 34]. A high degree of similarity between the fingerprints of a source and a suspicious model indicates that one is likely a derivative of the other.
- **Targeted Fingerprinting:** This paradigm seeks to construct a specific set of query-response pairs that are unique to a source model and its derivatives [11, 17, 44]. The key idea is that these special pairs act as a fingerprint, and only models from the source’s lineage will be able to consistently reproduce the predefined target responses. To perform an audit, the auditor submits these optimized queries to a suspicious model and checks if its outputs match the target responses. If a significant number of responses match, the model will be flagged as a likely derivative.

Despite the applicability of black-box methods, the empirical results from [37] highlight a significant performance gap between the two paradigms. White-box methods demonstrate remarkable effectiveness and reliability due to their direct access to the model’s stable and unique parameter space. Conversely, black-box techniques are found to be less reliable, as they solely depend on the outputs that are inherently more stochastic and less stable. This discrepancy underscores that reliably inferring a model’s lineage with only black-box access remains challenging and highlights the significance of designing more effective solutions.

3 Motivation and Theoretical Analysis

The remarkable effectiveness of white-box fingerprinting, especially static methods that achieve near-perfect auditing performance by directly analyzing model parameters [37], offers a crucial insight.

This success suggests that model parameters may be the most reliable features for fingerprinting. The rationale is that the immense scale of LLM parameters causes the static weights of independently developed models to become inherently unique identifiers.

In the black-box scenario, therefore, the key challenge is to *extract features that retain as much information about the model’s parameters as possible*. In this section, we analyze two features observable under black-box conditions: the model output Y and the gradient (or the differential) $D = dY/dX$. Existing black-box methods primarily rely on the direct observation of model outputs. However, our theoretical analysis reveals that observing the gradient D contains approximately more information about the parameter W than the output Y . This finding indicates that D is likely a more reliable feature for fingerprinting, forming the theoretical foundation for our ZEROPRINT.

3.1 The Nonlinearity Bottleneck: A Motivation Example

To illustrate why observing the output Y may yield less information about the model parameter W , let us consider a simple, fundamental unit commonly found in neural networks and LLMs:

$$Y = f(WX + K), \quad (2)$$

where X and Y represent the input and output, respectively, while W and K are the weight and bias. The function f is a non-linear activation function. An entire neural network, including LLM, is constructed by stacking a vast number of these non-linear units. In this example, the gradient of the output Y with respect to the input X can be expressed as:

$$D = \frac{dY}{dX} = Wf'(WX + K). \quad (3)$$

By analyzing both Y and D , a key difference emerges. If we observe the gradient D , for a fixed input X , the term $f'(WX + K)$ becomes a constant value. Consequently, analyzing the characteristics of D is effectively equivalent to analyzing the characteristics of the parameter W . In contrast, if we solely observe the output Y , the parameter W must pass through the non-linear function f . Typically, irreversible non-linear functions (*e.g.*, GELU and SiLU [15]) could compress the information, which can lead to a loss of information regarding W .

This observation leads us to a critical hypothesis: *Is taking the gradient D as a fingerprint a superior choice for preserving information about the parameter W compared to taking the output Y ?* In the following section, we will analyze this hypothesis from the perspective of information theory.

3.2 The Theoretical Edge: Why Gradients Carry More Information

In this section, we provide a theoretical analysis of the hypothesis proposed in the preceding section. To formally quantify and compare the amount of information that an observable feature contains about a model parameter, we employ the concept of *Fisher Information* from information theory [23]. Mathematically, Fisher Information quantifies how much an observable random variable X carries about an unknown parameter θ . A larger Fisher information

value implies more information, which can in turn lead to a more precise estimate. We begin with the formal definition:

DEFINITION 1 (FISHER INFORMATION). Let $\{p_\theta(x)\}_{\theta \in \Theta}$ be a family of probability density functions parameterized by θ . Assume $p_\theta(x)$ is differentiable with respect to θ and satisfies the regularity conditions that allow interchange of differentiation and integration. The **Fisher information** of a single observation X about the parameter θ is defined as

$$\mathcal{I}_X(\theta) := \mathbb{E}_{X \sim p_\theta} \left[\left(\frac{\partial}{\partial \theta} \log p_\theta(X) \right)^2 \right], \quad (4)$$

where the expectation is taken with respect to $p_\theta(x)$.

Based on this definition, to analyze whether observation D or observation Y contains more information about the model parameter W , it is equivalent to comparing their respective Fisher Information with regard to the parameter W . Through this analysis, we arrive at the following theorem:

THEOREM 1. Let $Y = f(WX + K)$, where f is neither a linear nor an affine function, W and K are constants with $W \neq 0$, and $D = dY/dX = Wf'(WX + K)$. Suppose that X follows a zero-mean normal distribution, denoted as $X \sim \mathcal{N}(0, \sigma_X^2)$, and $f''(K) \neq 0$. Under the first-order Taylor approximation, the Fisher information of Y and D with respect to W satisfies:

$$\mathcal{I}_D(W) \geq \left(\frac{c_1^2}{2W^2 c_2^2 \sigma_X^2} + 4 \right) \cdot \mathcal{I}_Y(W), \quad (5)$$

where $c_1 = f'(K)$, $c_2 = f''(K)$.

The proof is in Appendix A. Theorem 1 provides the mathematical proof for our hypothesis. The coefficient $c_1^2/(2W^2 c_2^2 \sigma_X^2) + 4 > 1$, leading to $\mathcal{I}_D(W) > \mathcal{I}_Y(W)$. The result formally establishes that the gradient D contains approximately more information about the parameter W than the output Y does. This theoretical advantage confirms that the gradient is an inherently more distinctive and reliable feature for a robust model fingerprint.

4 Methodology

The theoretical analysis in the previous section provides strong evidence that gradients are more informative features for LLM fingerprinting than outputs. However, in a practical black-box scenario, we lack the direct access required for backpropagation to compute these gradients precisely. To bridge this gap, we introduce ZEROPRINT, a novel black-box LLM fingerprinting method that leverages zeroth-order gradient estimation to approximate this information-rich feature. In the following sections, we will first define the threat model under which our method operates and then present a detailed introduction to the ZEROPRINT framework.

4.1 Threat Model

In this paper, we consider a typical and strict threat model for black-box LLM fingerprinting, which involves two primary parties: the copyright auditor, who aims to protect their intellectual property, and a potential adversary, who seeks to infringe upon it.

Adversary's Assumptions. The adversary's primary motivation is to leverage a SOTA, copyrighted model to develop their own

service with minimal investment. We assume the adversary obtains an illicit copy or an unauthorized derivative of a source model and deploys it for commercial use. The adversary possesses full control over this suspicious model and may perform modifications (e.g., fine-tuning on a custom dataset or altering its behavior with different system prompts) to either adapt it for a specific task or attempt to obfuscate its origin.

Auditor's Assumptions. The auditor operates under a set of constraints typical of a realistic black-box setting. We assume the auditor has the following capabilities:

- The auditor has only black-box, API-level access to both the source and suspicious models. This access is limited to providing inputs and observing the corresponding textual outputs. Internal states and output logits are inaccessible in our threat model.
- The auditor's interactions with the source and suspicious LLMs are bound by a query limit to ensure practicality and mimic real-world API usage constraints. We assume the auditor can send a maximum of q queries to a given LLM.

4.2 Overview of ZEROPRINT

Our ZEROPRINT is structured as a three-step process, designed to effectively generate and compare model fingerprints in a black-box setting. The overall workflow is shown in Figure 1, as follows:

- **Step 1: Query Set Construction.** The process begins with the construction of a query set, denoted by Q . This set comprises N input prompts $\{x_1, x_2, \dots, x_N\}$ that are designed to elicit distinctive responses from the models being audited.
- **Step 2: Fingerprint Generation.** The query set Q is then fed into an LLM $\mathcal{M}$ to obtain a corresponding set of outputs, Y . Using these input-output pairs, we employ a zeroth-order gradient estimation technique to approximate the model's Jacobian matrix for each input. The (aggregated) estimated Jacobian matrix, $J_{\mathcal{M}}$, serves as the unique fingerprint of the LLM. Given the source model $\mathcal{M}_o$ and the suspicious model $\mathcal{M}_s$, we extract their fingerprints F_o, F_s as follows:

$$F = \text{Extract}(\mathcal{M}(Q), Q) \triangleq J \approx \frac{\partial \mathcal{M}(Q)}{\partial Q}, \quad (6)$$

where $\mathcal{M}$ could be either $\mathcal{M}_o$ or $\mathcal{M}_s$.

- **Step 3: Fingerprint Comparison.** Finally, the similarity between the fingerprint of the source model, J_o , and that of the suspicious model, J_s , is calculated to determine provenance. In our proposed ZEROPRINT, we utilize the *Pearson correlation coefficient*, which can be interpreted as a form of normalized cosine similarity, as the primary metric. If the similarity score exceeds a predefined threshold τ , the suspicious model is flagged as a derivative of the source model.

$$\text{Sim}(J_o, J_s) = \text{Pearson}(J_o, J_s). \quad (7)$$

The core design of the ZEROPRINT method hinges on two critical components: the construction of an effective query set and the specific algorithm used for fingerprint generation. A central challenge in this process is adapting gradient estimation for a black-box setting where inputs and outputs are discrete text.

In a conventional continuous domain, zeroth-order optimization estimates the gradient of a function $f : \mathbb{R}^{\zeta} \rightarrow \mathbb{R}^{\eta}$ by observing

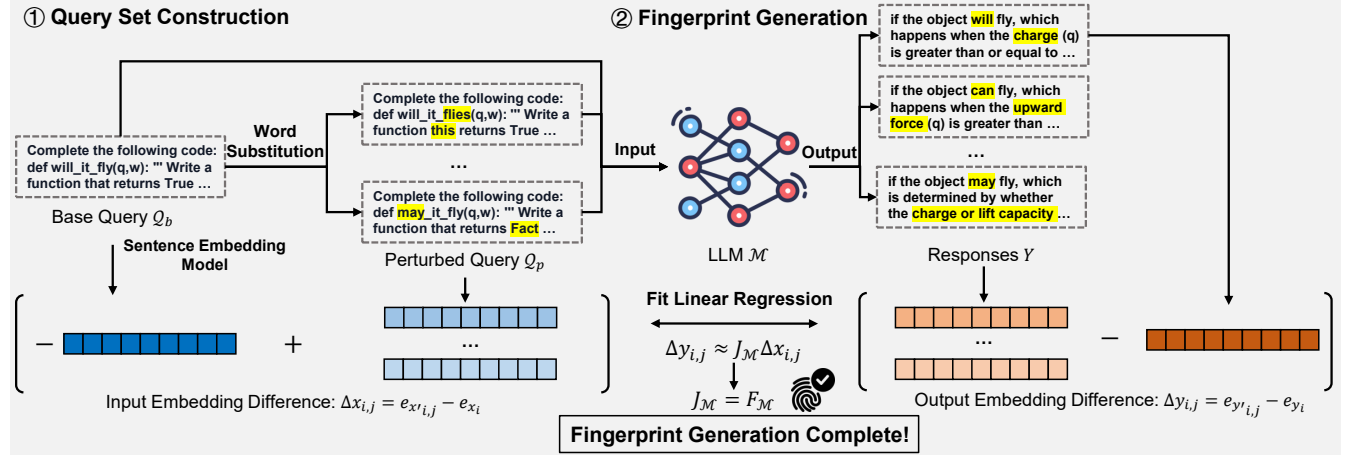


Figure 1: The pipeline of ZEROPRINT. The process starts with creating a query set by generating semantically similar “Perturbed Queries” from a “Base Query” through word substitution. Both query types are fed into the LLM to get responses. All inputs and outputs are converted to embeddings to calculate input and output difference vectors. Finally, a linear regression model is fitted on these differences to estimate the Jacobian matrix, which serves as the model’s unique fingerprint.

its output values. This is typically done by sampling a set of m random direction vectors $\{u_i\}_{i=1}^m$ and querying the function at points perturbed along these directions, $f(x + \delta u_i)$, where δ is a small smoothing parameter. The Jacobian matrix J (i.e., gradient) can then be approximated by solving a least-squares problem:

$$\min_J \sum_{i=1}^m \|(f(x + \delta u_i) - f(x)) - J(\delta u_i)\|_2^2. \quad (8)$$

However, this approach is not directly applicable in our black-box scenario. The input and output spaces are discrete and symbolic (i.e., text), not continuous vector spaces. It is not meaningful to add a small random vector to an input sentence. To overcome this, instead of adding continuous noise, ZEROPRINT generates perturbed inputs by performing semantic-preserving word substitutions on the base queries. We elaborate on the details in the following subsections.

4.3 Query Set Construction

The process of constructing the query set Q is composed of two main substeps. First, we select n base queries $Q_b = \{x_1, \dots, x_n\}$ designed for stability across a wide range of models. Second, for each base query x_i , we generate a corresponding set of m perturbed queries $Q_p^{(i)} = \{x'_{i,1}, \dots, x'_{i,m}\}$ using a semantic-preserving word substitution technique, which is essential for the subsequent gradient estimation step. In total, we generate $n \times m + n = N$ samples as the Query Set Q .

Base Query Selection. A key consideration in designing a generalizable fingerprinting method is the varying instruction-following capabilities across different LLM instances, especially between pre-trained base models and their fine-tuned derivatives. Base models, for instance, often struggle to produce coherent replies to complex queries, such as those used in QA tasks. To ensure our method elicits consistent and meaningful behavior from diverse LLMs, the selected queries could align with the fundamental objective of their pre-training, which is essentially a completion task. Therefore, we utilize a code completion task to form our base query set. This

approach is broadly compatible with models regardless of their fine-tuning stage. Specifically, we extract code snippets from a standard code generation dataset (e.g., HumanEval [3]) and prepend them with a simple instruction: “Complete the following code: [code snippet]”. From the dataset, we sample n such code snippets to create our set of base queries.

Constructing Perturbed Queries via Word Substitution. With the set of base queries, the next step is to generate perturbed samples for each one, which will be used to estimate the Jacobian. For each base query, we generate n_p perturbed counterparts using a word substitution method grounded in semantic similarity. The process leverages a pre-trained word embedding model (e.g., GloVe [30]) to find suitable replacements. For a given base query, we begin by randomly selecting r replaceable words within the base query. For each of these r words, we query the word embedding model to retrieve the top- k most semantically similar words. A new, perturbed query is then formed by randomly replacing the original word with one of these k candidates. This procedure is repeated independently to generate m unique perturbed samples for each base query, resulting in a total of $n \times m$ perturbed queries. The example and the pseudocode can be found in Appendix B.

4.4 Fingerprint Generation via Zeroth-order Gradient Estimation

Once the query set is constructed, the next step is to generate the LLM’s fingerprint. This process involves three substeps: (1) querying the target LLM, (2) embedding the inputs and outputs into a continuous vector space, and (3) applying a regression method to estimate the Jacobian matrix, which serves as the final fingerprint.

Querying the Target LLM. The procedure begins by feeding the base queries $\{x_i\}_{i=1}^n$ and their corresponding perturbed counterparts $\{x'_{i,j}\}_{j=1}^m$ to the target LLM, $\mathcal{M}$. To mitigate the inherent randomness of LLM outputs, each query is sent to the model t times, yielding a set of textual responses.

Table 1: Overall performance comparison between ZEROPRINT and baseline LLM fingerprinting methods.

Metric	White-box		Black-box			
	REEF	LLMmap	MET	SEF	TRAP	ZEROPRINT (Ours)
AUC $\uparrow$	0.896(± 0.021)	0.632(± 0.009)	0.661(± 0.003)	0.581(± 0.008)	0.712(± 0.027)	0.720 (± 0.013)
pAUC $\uparrow$	0.832(± 0.019)	0.635(± 0.009)	0.658(± 0.003)	0.646(± 0.004)	0.625(± 0.006)	0.683 (± 0.005)
TPR@1%FPR $\uparrow$	0.634(± 0.021)	0.253(± 0.032)	0.329(± 0.004)	0.282(± 0.012)	0.227(± 0.020)	0.366 (± 0.009)
MD $\uparrow$	2.091(± 0.058)	0.521(± 0.021)	1.457 (± 0.004)	0.172(± 0.014)	1.382(± 0.064)	1.457 (± 0.015)

Embedding Inputs and Outputs. We then employ a pre-trained sentence embedding model to transform all inputs and outputs into high-dimensional vectors. For each query, the final output embedding is computed by averaging the embeddings of the t responses, resulting in a stable representation. Let $E(\cdot)$ be the sentence embedding function. The embedded vector for the base input x_i is $e_{x_i} = E(x_i)$, and its averaged output vector is $\bar{e}_{y_i}$. Similarly, the vectors for a perturbed query are $e_{x'_{i,j}} = E(x'_{i,j})$ and $\bar{e}_{y'_{i,j}}$.

Estimate the Jacobian Matrix. With these vectors, we can formulate the Jacobian estimation as a linear regression problem. For each base query x_i , we compute the input and output difference vectors for all m of its perturbations:

$$\begin{aligned} \text{Input difference: } \Delta x_{i,j} &= e_{x'_{i,j}} - e_{x_i}, \\ \text{Output difference: } \Delta y_{i,j} &= \bar{e}_{y'_{i,j}} - \bar{e}_{y_i}. \end{aligned} \quad (9)$$

We seek to find a local Jacobian matrix J_i that best models the linear relationship $\Delta y_{i,j} \approx J_i \Delta x_{i,j}$. Given the practical constraints on the number of queries allowed, the number of perturbation samples m is often small relative to the dimensionality of the Jacobian matrix. This can lead to an underdetermined system, making an ordinary least squares estimate unstable. To address this, we use Ridge Regression, which introduces an L2 regularization term to produce a more robust estimate of the Jacobian:

$$J_i = \arg \min_J \sum_{j=1}^m \|\Delta y_{i,j} - J \Delta x_{i,j}\|_2^2 + \alpha \|J\|_F^2, \quad (10)$$

where α is the regularization hyperparameter and $\|\cdot\|_F$ is the Frobenius norm. Finally, a Jacobian matrix J_i is computed for each of the n base queries. The overall fingerprint for the model $\mathcal{M}$, denoted as $J_{\mathcal{M}}$, is the element-wise average of these individual Jacobian matrices. This aggregation step creates a more comprehensive and stable fingerprint that captures the model's characteristic behavior across a diverse set of inputs: $J_{\mathcal{M}} = \frac{1}{n} \sum_{i=1}^n J_i$.

With the final fingerprints generated for the source and suspicious models, the comparison is performed by calculating their similarity using Eq. (7) to determine copyright attribution.

5 Evaluation

5.1 Experimental Settings

Benchmark Models. The primary benchmark models in our experiments are from LEAFBENCH [37]. LEAFBENCH is built upon 7 mainstream pre-trained foundation models, including variants of Qwen [42], Llama [10, 43], Mistral [16], Gemma [40], and TinyLlama [55]. These models were selected with parameter counts ranging from 1.1B to 14B. LEAFBENCH also implements various LLM

post-development techniques, including fine-tuning, model merging, distillation, different system prompts, RAG, adversarial attacks, etc. In total, LEAFBENCH provides 149 different model instances and establishes a representative benchmark for evaluating fingerprint-based LLM copyright auditing. The details of LEAFBENCH can be found in Appendix B.

Baselines. For the baseline methods, we implement 3 untargeted black-box methods, LLMmap [29], Model Equality Testing (MET) [9], and Sentence Embedding Fingerprinting (SEF) [37], and 1 targeted black-box method, TRAP [11]. We also include a white-box method, REEF [54], for reference. Note that targeted methods, including TRAP, are typically *semi-black-box*. They assume black-box access to the suspicious model, but white-box access to the source model, which is a slightly stronger assumption than untargeted methods. The maximum number of queries is set to $q = 200$.

Settings of ZEROPRINT. For default settings, we select $n = 2$ base queries from HumanEval [3] and generate $m = 4$ perturbed queries, with a total number of 10 queries. We input each query to the LLM $t = 20$ times, leading to 200 queries in total. For the sentence embedding model, we choose all-mpnet-base-v2 [32], which is a classic yet powerful model also used in LLMmap. More detailed settings are in Appendix B.

Metrics. In our experiments, following prior work [37], we primarily leverage two metrics: (1) Area Under the ROC Curve (AUC) and (2) Partial AUC with FPR $\in [0, 0.05]$ (pAUC). The AUC provides a holistic measure of a method's ability to distinguish between related model pairs (a source model and its derivative) and unrelated pairs across all possible classification thresholds τ . In contrast, the pAUC metric is specifically designed to assess performance by focusing on a specific low false-positive-rate (FPR) region (*i.e.*, $[0, 0.05]$) of the ROC curve. Higher AUC and pAUC scores indicate superior performance of a fingerprinting method.

5.2 Main Results

Table 1 presents the overall performance comparison between ZEROPRINT and various baseline methods. In addition to the primary metrics of AUC and pAUC, we also report the True Positive Rate at a 1% False Positive Rate (TPR@1%FPR) and the Mahalanobis Distance (MD). The TPR@1%FPR metric measures the true positive rate that a method can achieve while maintaining a low FPR of 1%. The MD metric measures the separability of the fingerprint distributions, reflecting the distance between positive pairs (a source model and its derivatives) and negative pairs (independent models). A larger MD indicates a stronger ability to distinguish between related and unrelated models. For both TPR@1%FPR and MD, higher values signify better performance.

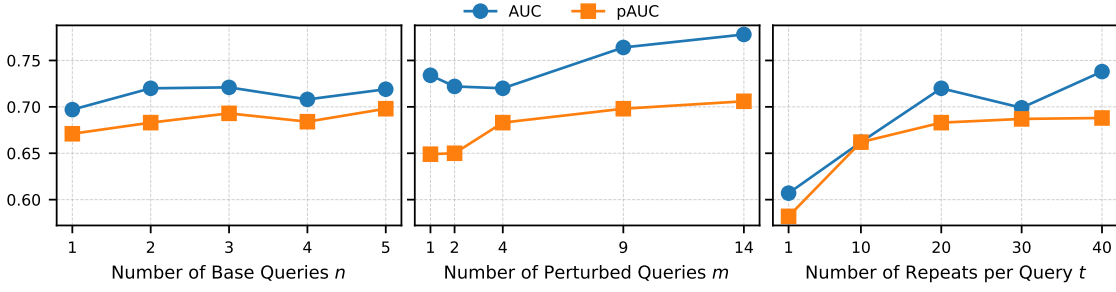


Figure 2: The performance of ZEROPRINT with different numbers of queries. We study three factors: the number of base queries n , the number of perturbed queries m , and the number of repeats per query t .

The results demonstrate that ZEROPRINT achieves SOTA or comparable performance across all evaluated metrics. It significantly outperforms existing untargeted methods. Notably, ZEROPRINT also shows superior performance compared to the targeted, semi-black-box method TRAP, improving the AUC to 0.720 and the pAUC to 0.683. However, a performance gap remains when compared to white-box methods like REEF, which have direct access to model internals. It is worth noting that these results were achieved under a strict query limit of 200 queries per model. Subsequent experiments in our ablation study reveal that ZEROPRINT’s performance can be further enhanced by increasing the number of queries allowed. This scalability is further detailed in Section 5.3.

5.3 Ablation Study

Effects of Different Numbers of Queries. The total number of queries is a critical factor influencing the performance of ZEROPRINT. This number is determined by three key hyperparameters: the number of base queries (n), the number of perturbed queries generated for each base query (m), and the number of repeated generations for each query (t). To understand their impact, we conducted a series of experiments, varying one parameter at a time while keeping others fixed. The results are illustrated in Figure 2.

- **Effects of Different Numbers of Base Queries n .** We first investigated the impact of n by setting its value to 1, 2, 3, 4, and 5, with $n = 2$ being the default in our main experiments. As shown in the first subplot of Figure 2, the performance of ZEROPRINT remains relatively stable across different values of n . The AUC and pAUC fluctuate in a small interval. This observation suggests that the number of base queries may not have a significant impact on the effectiveness of ZEROPRINT. A small number of base queries (*e.g.*, 2) is sufficient to generate a distinctive fingerprint.
- **Effects of Different Numbers of Perturbed Queries m .** Next, we examined the influence of m with values set to 1, 2, 4, 9, and 14. The default value used in our main experiments was $m = 4$. The results, depicted in the second subplot of Figure 2, reveal that m has a pronounced effect on the method’s performance. When m is small (*e.g.*, 1 or 2), the estimated gradients are less stable, leading to inconsistent results (*e.g.*, a high AUC but a low pAUC). However, as m increases to 4 and beyond, both AUC and pAUC show a clear and significant upward trend. The performance peaks at $m = 14$, achieving the highest AUC (0.778) and pAUC (0.706) in this set of experiments. This trend highlights the potential of ZEROPRINT to achieve even greater effectiveness and

Table 2: The performance of ZEROPRINT using base queries from different datasets.

Dataset	TruthfulQA	SQuAD	DROP	Wikipedia	HumanEval (Ours)
AUC $\uparrow$	0.622	0.665	0.676	0.698	0.720
pAUC $\uparrow$	0.668	0.666	0.651	0.647	0.683

robustness with an increased query budget, as more perturbed queries provide a more accurate gradient estimation.

- **Effects of Different Numbers of Repeats per Query t .** Finally, we analyzed the effect of t , testing values of 1, 10, 20, 30, and 40, with $t = 20$ as the default setting. This parameter is designed to mitigate the inherent randomness in LLM-generated outputs. The third subplot of Figure 2 shows that when t is small, performance suffers due to this stochasticity, yielding an AUC of only 0.607 for $t = 1$. As t increases, the performance improves substantially, with the AUC reaching 0.720 at $t = 20$. Beyond this point, the results begin to plateau and fluctuate slightly. This indicates that at $t = 20$, the averaged output embedding for a given input becomes sufficiently stable. Further increases in t offer diminishing returns.

Effects of Different Query Data. The choice of data for constructing base queries is another crucial element. We evaluated the performance of ZEROPRINT using base queries derived from five different datasets: HumanEval [3] (our default), three question-answering (QA) datasets (TruthfulQA [21], SQuAD [31], and DROP [7]), and one text completion dataset (Wikipedia [8]). The results, presented in Table 2, show that the source of the query data significantly impacts fingerprinting effectiveness. When using queries from the QA datasets, the performance was consistently poor. Similarly, the Wikipedia dataset, despite being a completion task, also failed to achieve comparable results to HumanEval. This can be attributed to the nature of the tasks. Pure text completion is inherently more diverse and less structured than code completion. Thus, various derivative models tend to exhibit less consistent behavior on text-based prompts, making it harder to extract a stable and reliable fingerprint. In contrast, the structured nature of code completion elicits more uniform responses, leading to better performance.

Effects of Different Sentence Embedding Models. We investigated the influence of the sentence embedding model on ZEROPRINT’s performance. We experimented with five different models of varying scales and embedding dimensions (details in Appendix B). The results, detailed in Table 3, reveal a clear trade-off related to the

Table 3: The performance of ZEROPRINT using different sentence embedding models. “Dim” signifies the dimension of the generated sentence embeddings.

Dataset	Model2Vec	MiniLM	MPNet (main)	EmbeddingGemma	Qwen3
Dim	256	384	768	768	1024
AUC $\uparrow$	0.683	0.702	0.720	0.721	0.698
pAUC $\uparrow$	0.659	0.682	0.683	0.682	0.659

Table 4: The fingerprint similarities between the original instruct LLM and its attacked version using ZEROPRINT.

Attack	Input Paraphrasing			Output Perturbation		
Setting	0.5B	1B	2B	0.05	0.10	0.15
Qwen2.5-7B	0.878	0.876	0.881	0.865	0.894	0.868
Llama3.1-8B	0.875	0.913	0.904	0.887	0.868	0.903

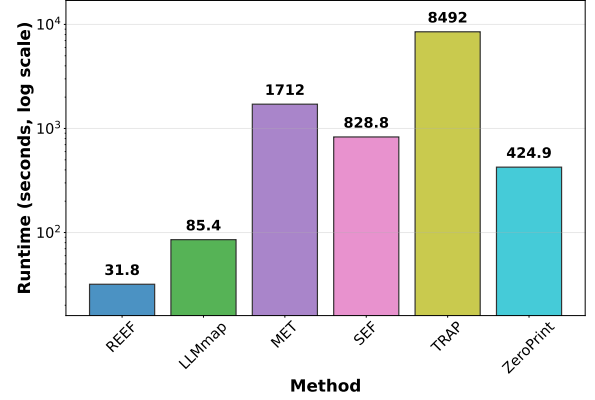
embedding dimension. When the dimension is too small, the model fails to adequately capture the semantic nuances of the sentences, leading to poor performance. Conversely, a larger embedding dimension increases the size of the Jacobian matrix to be estimated. Given the constraints on the number of queries, estimating this larger matrix becomes less accurate, which also degrades the overall fingerprinting effectiveness. This is supported by the similar performance of MPNet and EmbeddingGemma, which share the same embedding dimension and achieve comparable results.

5.4 Resistance to Potential Adaptive Attacks

To assess the robustness of ZEROPRINT, we evaluated its resilience against two potential adaptive attack strategies an adversary might employ to evade detection:

- **Input Paraphrasing Attack:** This attack attempts to obfuscate queries by rewriting them using a separate, smaller LLM before submitting them to the target model. We simulated this using three paraphrasing models of varying sizes (0.5B, 1B, and 2B).
- **Output Perturbation Attack:** This attack involves adding noise to the model’s output logits to disrupt the fingerprinting process. We tested this by introducing Gaussian noise with three different standard deviations (0.05, 0.10, and 0.15).

Detailed settings can be found in Appendix B. The experimental results, summarized in Table 4, demonstrate that ZEROPRINT maintains high fingerprint similarity under both attack scenarios, confirming its strong resilience. The robustness against the input paraphrasing attack stems from our method’s reliance on the semantic difference between embeddings. While paraphrasing alters the surface form of the text, the core semantic relationship and vector difference between the base and perturbed queries remain relatively stable, allowing for a reliable estimation of the Jacobian matrix. For the output perturbation attack, ZEROPRINT’s resilience can be attributed to two key factors. First, averaging the embeddings from multiple queries effectively filters out random, zero-mean noise. Second, since our fingerprint is derived from the estimated gradient, which is the relationship between input and output changes, it is inherently less sensitive to small, random perturbations in the absolute output values, thus preserving the fingerprint’s integrity.

**Figure 3: The overhead of different fingerprinting methods.**

5.5 Overhead Evaluation

To evaluate practical efficiency, we measured the total time for fingerprint extraction and verification on a subset of four LLMs. Experimental details can be found in Appendix B. From Figure 3, ZEROPRINT demonstrates a competitive overhead of 424.9 seconds, which is significantly faster than most black-box baselines like MET, SEF, and TRAP. Its runtime is only greater than the white-box REEF and LLMmap, the latter of which uses a minimal set of only 8 queries. Given that the fingerprinting for a single model can be completed in under two minutes on average, ZEROPRINT’s process completes in an acceptable timeframe. Overall, ZEROPRINT is a computationally practical solution for real-world auditing.

6 Conclusion

In this paper, we addressed the critical challenge of protecting the intellectual property of large language models (LLMs) through fingerprinting, a technique that aims to identify a model’s origin by extracting a unique, intrinsic signature. We began by formally demonstrating, through Fisher Information Theory, that a model’s gradient contains significantly more information about its parameters than its output, establishing it as a superior feature for fingerprinting. Building on this theoretical foundation, we introduced ZEROPRINT, a novel black-box fingerprinting method that successfully approximates these information-rich gradients. By simulating input perturbations through semantic-preserving word substitutions, ZEROPRINT overcomes the inherent challenges of applying gradient estimation to the discrete domain of text. Our extensive experiments show that ZEROPRINT not only achieves SOTA performance, outperforming existing black-box methods in both effectiveness and robustness, but also operates with a practical overhead. This work represents a significant step forward in developing reliable and secure mechanisms for LLM copyright auditing.

Acknowledgments

This research is supported in part by the “Pioneer” and “Leading Goose” R&D Program of Zhejiang (2024C01169), the Kunpeng-Ascend Science and Education Innovation Excellence/Incubation Center, and the National Natural Science Foundation of China under Grants (62441238, U2441240).

References

- [1] Nicholas Carlini, Daniel Paleka, Krishnamurthy Dvijotham, Thomas Steinke, Jonathan Hayase, A Feder Cooper, Katherine Lee, Matthew Jagielski, Milad Nasr, Arthur Conmy, et al. 2024. Stealing part of a production language model. In *International Conference on Machine Learning*. 5680–5705.
- [2] Jialuo Chen, Jingyi Wang, Tinglan Peng, Youcheng Sun, Peng Cheng, Shouling Ji, Xingjun Ma, Bo Li, and Dawn Song. 2022. Copy, right? a testing framework for copyright protection of deep learning models. In *IEEE Symposium on Security and Privacy*. 824–841.
- [3] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, et al. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374* (2021).
- [4] Pin-Yu Chen, Huan Zhang, Yash Sharma, Jinfeng Yi, and Cho-Jui Hsieh. 2017. Zoo: Zeroth order optimization based black-box attacks to deep neural networks without training substitute models. In *ACM Workshop on Artificial Intelligence and Security*. 15–26.
- [5] Gheorghe Comanici, Eric Bieber, Mike Schaekermann, Ice Pasupat, Naveen Sachdeva, Inderjit Dhillon, Marcel Blistein, Ori Ram, Dan Zhang, Evan Rosen, et al. 2025. Gemini 2.5: Pushing the frontier with advanced reasoning, multi-modality, long context, and next generation agentic capabilities. *arXiv preprint arXiv:2507.06261* (2025).
- [6] Yang Deng, An Zhang, Yankai Lin, Xu Chen, Ji-Rong Wen, and Tat-Seng Chua. 2024. Large language model powered agents in the web. In *The ACM Web Conference*. 1242–1245.
- [7] Dheeru Dua, Yizhong Wang, Pradeep Dasigi, Gabriel Stanovsky, Sameer Singh, and Matt Gardner. 2019. DROP: A Reading Comprehension Benchmark Requiring Discrete Reasoning Over Paragraphs. In *Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*.
- [8] Wikimedia Foundation. [n. d.]. *Wikimedia Downloads*. <https://dumps.wikimedia.org>
- [9] Irena Gao, Percy Liang, and Carlos Guestrin. 2025. Model Equality Testing: Which Model Is This API Serving?. In *International Conference on Learning Representations*.
- [10] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, et al. 2024. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783* (2024).
- [11] Martin Gubri, Dennis Ulmer, Hwaran Lee, Sangdoo Yun, and Seong Joon Oh. 2024. TRAP: Targeted Random Adversarial Prompt HoneyPot for Black-Box Identification. In *Findings of the Association for Computational Linguistics: ACL 2024*. 11496–11517.
- [12] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, et al. 2025. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *Nature* (2025), 633–638.
- [13] Yiling He, Hongyu She, Xingzhi Qian, Xinran Zheng, Zhuo Chen, Zhan Qin, and Lorenzo Cavallaro. 2025. On Benchmarking Code LLMs for Android Malware Analysis. In *ACM SIGSOFT International Symposium on Software Testing and Analysis Workshop*.
- [14] Dan Hendrycks, Collin Burns, Steven Basart, Andrew Critch, Jerry Li, Dawn Song, and Jacob Steinhardt. 2021. Aligning AI With Shared Human Values. In *International Conference on Learning Representations*.
- [15] Dan Hendrycks and Kevin Gimpel. 2016. Gaussian error linear units (gelus). *arXiv preprint arXiv:1606.08415* (2016).
- [16] Albert Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, et al. 2023. Mistral 7B. *arXiv preprint arXiv:2310.06825* (2023).
- [17] Heng Jin, Chaoyu Zhang, Shanghao Shi, Wenjing Lou, and Y Thomas Hou. 2024. Profiling: A fingerprinting-based intellectual property protection scheme for large language models. In *IEEE Conference on Communications and Network Security*. IEEE, 1–9.
- [18] Qiao Jin, Bhuwan Dhingra, Zhengping Liu, William Cohen, and Xinghua Lu. 2019. PubMedQA: A Dataset for Biomedical Research Question Answering. In *Conference on Empirical Methods in Natural Language Processing and International Joint Conference on Natural Language Processing*. 2567–2577.
- [19] Yiming Li, Shuo Shao, Yu He, Junfeng Guo, Tianwei Zhang, Zhan Qin, Pin-Yu Chen, Michael Backes, Philip Torr, Dacheng Tao, and Kui Ren. 2025. Rethinking Data Protection in the (Generative) Artificial Intelligence Era. *arXiv preprint arXiv:2507.03034* (2025).
- [20] Zongjie Li, Chaozheng Wang, Shuai Wang, and Cuiyun Gao. 2023. Protecting intellectual property of large language model-based code generation apis via watermarks. In *ACM SIGSAC Conference on Computer and Communications Security*. 2336–2350.
- [21] Stephanie Lin, Jacob Hilton, and Owain Evans. 2022. TruthfulQA: Measuring How Models Mimic Human Falsehoods. In *Annual Meeting of the Association for Computational Linguistics*. 3214–3252.
- [22] Zhifan Luo, Shuo Shao, Su Zhang, Lijing Zhou, Yuke Hu, Zhihao Liu, and Zhan Qin. 2026. Shadow in the Cache: Unveiling and Mitigating Privacy Risks of KV-cache in LLM Inference. In *Network and Distributed System Security Symposium*.
- [23] Alexander Ly, Maarten Marsman, Josine Verhagen, Raoul PPP Grasman, and Eric-Jan Wagenmakers. 2017. A tutorial on Fisher information. *Journal of Mathematical Psychology* 80 (2017), 40–55.
- [24] Hope Elizabeth McGovern, Rickard Stureborg, Yoshi Suhara, and Dimitris Alikaniotis. 2025. Your Large Language Models are Leaving Fingerprints. In *International Conference on Computational Linguistics Workshop*. 85–95.
- [25] Daye Nam, Andrew Macvean, Vincent Hellendoorn, Bogdan Vasilescu, and Brad Myers. 2024. Using an llm to help with code understanding. In *IEEE/ACM International Conference on Software Engineering*. 1–13.
- [26] Liangbo Ning, Ziran Liang, Zhuohang Jiang, Haoqiao Qu, Yujuan Ding, Wenqi Fan, Xiao-yong Wei, Shanru Lin, Hui Liu, Philip S Yu, et al. 2025. A survey of webagents: Towards next-generation ai agents for web automation with large foundation models. In *ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 6140–6150.
- [27] Team OLMo. 2024. 2 OLMo 2 Furious. *arXiv preprint arXiv:2501.00656* (2024).
- [28] OpenAI. 2023. GPT-4 Technical Report. *arXiv preprint arXiv:2303.08774* (2023).
- [29] Dario Pasquini, Evgenios M Kornaropoulos, and Giuseppe Ateniese. 2025. Llmmap: Fingerprinting for large language models. In *USENIX Security Symposium*.
- [30] Jeffrey Pennington, Richard Socher, and Christopher D Manning. 2014. Glove: Global vectors for word representation. In *Conference on Empirical Methods in Natural Language Processing*. 1532–1543.
- [31] Pranav Rajpurkar, Jian Zhang, Konstantin Lopyrev, and Percy Liang. 2016. SQuAD: 100,000+ Questions for Machine Comprehension of Text. In *Conference on Empirical Methods in Natural Language Processing*. 2383–2392.
- [32] Nils Reimers and Iryna Gurevych. 2019. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. In *Conference on Empirical Methods in Natural Language Processing and International Joint Conference on Natural Language Processing*. Association for Computational Linguistics.
- [33] Kui Ren, Ziqi Yang, Li Lu, Jian Liu, Yiming Li, Jie Wan, Xiaodi Zhao, Xianheng Feng, and Shuo Shao. 2024. SoK: On the Role and Future of AIGC Watermarking in the Era of Gen-AI. *arXiv preprint arXiv:2411.11478* (2024).
- [34] Zhenzhen Ren, Guobiao Li, Sheng Li, Zhenxing Qian, and Xinpeng Zhang. 2025. CoTSRF: Utilize Chain of Thought as Stealthy and Robust Fingerprint of Large Language Models. *arXiv preprint arXiv:2505.16785* (2025).
- [35] Prateek Sancheti, Kamalakara Karlapalem, and Kavita Vemuri. 2024. Llm driven web profile extraction for identical names. In *The ACM Web Conference*.
- [36] Henrique Schechter Vera, Sahil Dua, Biao Zhang, Daniel Salz, Ryan Mullins, et al. 2025. EmbeddingGemma: Powerful and Lightweight Text Representations. *arXiv preprint arXiv:2509.20354* (2025).
- [37] Shuo Shao, Yiming Li, Yu He, Hongwei Yao, Wenyuan Yang, Dacheng Tao, and Zhan Qin. 2025. SoK: Large Language Model Copyright Auditing via Fingerprinting. *arXiv preprint arXiv:2508.19843* (2025).
- [38] Shuo Shao, Yiming Li, Hongwei Yao, Yiling He, Zhan Qin, and Kui Ren. 2025. Explanation as a Watermark: Towards Harmless and Multi-bit Model Ownership Verification via Watermarking Feature Attribution. In *Network and Distributed System Security Symposium*.
- [39] Mingjie Sun, Yida Yin, Zhiqiu Xu, J Zico Kolter, and Zhuang Liu. 2025. Idiosyncrasies in large language models. In *International Conference on Machine Learning*.
- [40] Gemma Team. 2024. Gemma 2: Improving open language models at a practical size. *arXiv preprint arXiv:2408.00118* (2024).
- [41] Qwen Team. 2024. Qwen2 technical report. *arXiv preprint arXiv:2407.10671* 2 (2024).
- [42] Qwen Team. 2024. Qwen2.5: A Party of Foundation Models. <https://qwenlm.github.io/blog/qwen2.5/>
- [43] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajwal Bhargava, Shrutu Bhosale, et al. 2023. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288* (2023).
- [44] Yun-Yun Tsai, Chuan Guo, Junfeng Yang, and Laurens van der Maaten. 2025. RoFL: Robust Fingerprinting of Language Models. *arXiv preprint arXiv:2505.12682* (2025).
- [45] Stephan Tulkens and Thomas van Dongen. 2024. Model2Vec: Fast State-of-the-Art Static Embeddings. <https://github.com/MinishLab/model2vec>
- [46] Tianhao Wang and Florian Kerschbaum. 2021. Riga: Covert and robust white-box watermarking of deep neural networks. In *The ACM Web Conference*. 993–1004.
- [47] Zehao Wu, Yanjie Zhao, and Haoyu Wang. 2025. Gradient-Based Model Fingerprinting for LLM Similarity Detection and Family Classification. *arXiv preprint arXiv:2506.01631* (2025).
- [48] Haoyi Xiong, Jiang Bian, Yuchen Li, Xuhong Li, Mengnan Du, Shuaiqiang Wang, et al. 2024. When search engine services meet large language models: visions and challenges. *IEEE Transactions on Services Computing* (2024).
- [49] Jiashu Xu, Fei Wang, Mingyu Ma, Pang Wei Koh, Chaowei Xiao, and Muhao Chen. 2024. Instructional Fingerprinting of Large Language Models. In *Conference of the North American Chapter of the Association for Computational Linguistics*.
- [50] Zhenhua Xu, Xubin Yue, Zhebo Wang, Qichen Liu, Xixiang Zhao, Jingxuan Zhang, Wenjun Zeng, Wengpeng Xing, Dezhang Kong, Changting Lin, et al.

2025. Copyright Protection for Large Language Models: A Survey of Methods, Challenges, and Trends. *arXiv preprint arXiv:2508.11548* (2025).
- [51] Hongwei Yao, Yun Xia, Shuo Shao, Haoran Shi, Tong Qiao, and Cong Wang. 2025. Black-Box Guardrail Reverse-engineering Attack. *arXiv preprint arXiv:2511.04215* (2025).
- [52] Boyi Zeng, Lizheng Wang, Yuncong Hu, Yi Xu, Chenghu Zhou, Xinbing Wang, Yu Yu, and Zhouhan Lin. 2024. Huref: Human-readable fingerprint for large language models. In *Annual Conference on Neural Information Processing Systems*.
- [53] Jie Zhang, Jiayuan Li, Haiqiang Fei, Lun Li, and Hongsong Zhu. 2024. EasyDetector: Using Linear Probe to Detect the Provenance of Large Language Models. In *IEEE International Conference on Trust, Security and Privacy in Computing and Communications*.
- [54] Jie Zhang, Dongrui Liu, Chen Qian, Linfeng Zhang, Yong Liu, Yu Qiao, and Jing Shao. 2025. Reef: Representation encoding fingerprints for large language models. In *International Conference on Learning Representations*.
- [55] Peiyuan Zhang, Guangtao Zeng, Tianduo Wang, and Wei Lu. 2024. Tinyllama: An open-source small language model. *arXiv preprint arXiv:2401.02385* (2024).
- [56] Yanzhao Zhang, Mingxin Li, Dingkun Long, Xin Zhang, et al. 2025. Qwen3 Embedding: Advancing Text Embedding and Reranking Through Foundation Models. *arXiv preprint arXiv:2506.05176* (2025).
- [57] Yue Zheng, Si Wang, and Chip-Hong Chang. 2022. A DNN fingerprint for non-repudiable model ownership identification and piracy detection. *IEEE Transactions on Information Forensics and Security* 17 (2022), 2977–2989.
- [58] Yaochen Zhu, Chao Wan, Harald Steck, Dawen Liang, Yesu Feng, Nathan Kallus, and Jundong Li. 2025. Collaborative Retrieval for Large Language Model-based Conversational Recommender Systems. In *The ACM Web Conference*. 3323–3334.
- [59] Andy Zou, Zifan Wang, Nicholas Carlini, Milad Nasr, J Zico Kolter, and Matt Fredrikson. 2023. Universal and transferable adversarial attacks on aligned language models. *arXiv preprint arXiv:2307.15043* (2023).

Appendix

A Proof of Theorem 1

Before proving Theorem 1, we first introduce and prove a related lemma about the Fisher information of a random variable following a normal distribution, as follows.

LEMMA 1 (FISHER INFORMATION FOR A NORMAL DISTRIBUTION). *Let X be a random variable following a normal distribution with mean $\mu(\theta)$ and variance $\sigma^2(\theta)$, which are differentiable functions of a parameter θ . The Fisher information of X with respect to θ is:*

$$\mathcal{I}_X(\theta) = \frac{1}{\sigma^2(\theta)} \left(\frac{\partial \mu(\theta)}{\partial \theta} \right)^2 + \frac{1}{2\sigma^4(\theta)} \left(\frac{\partial \sigma^2(\theta)}{\partial \theta} \right)^2. \quad (11)$$

PROOF OF LEMMA 1. The probability density function of the normal distribution is:

$$p(x; \theta) = \frac{1}{\sqrt{2\pi\sigma^2(\theta)}} \exp \left(-\frac{(x - \mu(\theta))^2}{2\sigma^2(\theta)} \right). \quad (12)$$

The log-likelihood function is:

$$\log p(x; \theta) = -\frac{1}{2} \log(2\pi) - \frac{1}{2} \log(\sigma^2(\theta)) - \frac{(x - \mu(\theta))^2}{2\sigma^2(\theta)}. \quad (13)$$

The score function is the partial derivative of the log-likelihood with respect to θ :

$$\frac{\partial}{\partial \theta} \log p(x; \theta) = \frac{(x - \mu(\theta))}{\sigma^2(\theta)} \frac{\partial \mu(\theta)}{\partial \theta} + \frac{(x - \mu(\theta))^2 - \sigma^2(\theta)}{2\sigma^4(\theta)} \frac{\partial \sigma^2(\theta)}{\partial \theta}. \quad (14)$$

The Fisher information is the expectation of the square of the score. Squaring the score function and taking the expectation, the cross-product term will have an expectation of 0. Thus, we only need to consider the expectation of the squared terms. Let $Z = X - \mu(\theta) \sim \mathcal{N}(0, \sigma^2(\theta))$, using the central moments of the normal distribution

$\mathbb{E}[Z] = 0$, $\mathbb{E}[(Z)^2] = \sigma^2(\theta)$, and $\mathbb{E}[(Z)^4] = 3\sigma^4(\theta)$, we get:

$$\mathcal{I}_X(\theta) = \mathbb{E} \left[\left(\frac{(Z)}{\sigma^2(\theta)} \frac{\partial \mu(\theta)}{\partial \theta} \right)^2 \right] + \mathbb{E} \left[\left(\frac{(Z)^2 - \sigma^2(\theta)}{2\sigma^4(\theta)} \frac{\partial \sigma^2(\theta)}{\partial \theta} \right)^2 \right]. \quad (15)$$

For the first term, we have:

$$\begin{aligned} & \mathbb{E} \left[\left(\frac{(Z)}{\sigma^2(\theta)} \frac{\partial \mu(\theta)}{\partial \theta} \right)^2 \right] \\ &= \frac{1}{\sigma^4(\theta)} \left(\frac{\partial \mu(\theta)}{\partial \theta} \right)^2 \mathbb{E}[(Z)^2] = \frac{1}{\sigma^2(\theta)} \left(\frac{\partial \mu(\theta)}{\partial \theta} \right)^2. \end{aligned} \quad (16)$$

For the second term, we have:

$$\begin{aligned} & \mathbb{E} \left[\left(\frac{(Z)^2 - \sigma^2(\theta)}{2\sigma^4(\theta)} \frac{\partial \sigma^2(\theta)}{\partial \theta} \right)^2 \right] \\ &= \frac{1}{4\sigma^8(\theta)} \left(\frac{\partial \sigma^2(\theta)}{\partial \theta} \right)^2 \mathbb{E}[(Z)^2 - \sigma^2(\theta)]^2 \\ &= \frac{1}{4\sigma^8(\theta)} \left(\frac{\partial \sigma^2(\theta)}{\partial \theta} \right)^2 (\mathbb{E}[(Z)^4] - 2\sigma^2(\theta)\mathbb{E}[(Z)^2] + \sigma^4(\theta)) \\ &= \frac{1}{4\sigma^8(\theta)} \left(\frac{\partial \sigma^2(\theta)}{\partial \theta} \right)^2 (3\sigma^4(\theta) - 2\sigma^4(\theta) + \sigma^4(\theta)) \\ &= \frac{2\sigma^4(\theta)}{4\sigma^8(\theta)} \left(\frac{\partial \sigma^2(\theta)}{\partial \theta} \right)^2 = \frac{1}{2\sigma^4(\theta)} \left(\frac{\partial \sigma^2(\theta)}{\partial \theta} \right)^2. \end{aligned} \quad (17)$$

Therefore, we get:

$$\mathcal{I}_X(\theta) = \frac{1}{\sigma^2(\theta)} \left(\frac{\partial \mu(\theta)}{\partial \theta} \right)^2 + \frac{1}{2\sigma^4(\theta)} \left(\frac{\partial \sigma^2(\theta)}{\partial \theta} \right)^2. \quad (18)$$

This completes the proof of Lemma 1. $\square$

Given Lemma 1, we then provide the proof of Theorem 1.

THEOREM 1. *Let $Y = f(WX + K)$, where f is neither a linear nor an affine function, W and K are constants with $W \neq 0$, and $D = dY/dX = Wf'(WX + K)$. Suppose that X follows a zero-mean normal distribution, denoted as $X \sim \mathcal{N}(0, \sigma_X^2)$, and $f''(K) \neq 0$. Under the first-order Taylor approximation, the Fisher information of Y and D with respect to W satisfies:*

$$\mathcal{I}_D(W) \geq \left(\frac{c_1^2}{2W^2\sigma_X^2} + 4 \right) \cdot \mathcal{I}_Y(W). \quad (19)$$

where $c_1 = f'(K)$, $c_2 = f''(K)$.

PROOF OF THEOREM 1. We first analyze the Fisher information of Y with respect to W . Let $Z = WX + K$. Since $X \sim \mathcal{N}(0, \sigma_X^2)$, the random variable Z is also normally distributed with mean $\mu_Z(W) = K$ and variance $\sigma_Z^2(W) = W^2\sigma_X^2$. We can now use Lemma 1 to calculate $\mathcal{I}_Z(W)$ with the parameter $\theta = W$. The partial derivatives of the mean and variance of Z with respect to W are:

$$\frac{\partial \mu(W)}{\partial W} = 0 \quad \text{and} \quad \frac{\partial \sigma^2(W)}{\partial W} = 2W\sigma_X^2. \quad (20)$$

Substituting these into the formula from Lemma 1:

$$\begin{aligned} \mathcal{I}_Z(W) &= \frac{1}{\sigma^2(W)} \left(\frac{\partial \mu(W)}{\partial W} \right)^2 + \frac{1}{2\sigma^4(W)} \left(\frac{\partial \sigma^2(W)}{\partial W} \right)^2 \\ &= \frac{1}{W^2 \sigma_X^2} * 0^2 + \frac{1}{2(W^2 \sigma_X^2)^2} (2W \sigma_X^2)^2 = \frac{4W^2 \sigma_X^4}{2W^4 \sigma_X^4} = \frac{2}{W^2}. \end{aligned} \quad (21)$$

Since $Y = f(Z)$ and f is independent of the parameter W , according to the Data Processing Inequality (DPI), the amount of information in data does not increase after any processing that is independent of the parameters. Therefore, we have:

$$\mathcal{I}_Y(W) \leq \mathcal{I}_Z(W) = \frac{2}{W^2}. \quad (22)$$

Then, we analyze the Fisher information of $D = Wf'(WX + K)$ with respect to W . Expand f' about $Z = K$:

$$f'(Z) = c_1 + c_2(Z - K) + R_2(Z), \quad (23)$$

where $c_1 := f'(K)$, $c_2 = f''(K)$, and $R_2(Z)$ is the Taylor remainder. Neglecting the remainder term R_2 (this is the first-order approximation), and using $Z - K = WX$, we obtain the approximate linear form as follows:

$$D \approx D_{\text{approx}} := c_1 W + c_2 W^2 X. \quad (24)$$

Under this approximation, D_{approx} follows a normal distribution $\mathcal{N}(\mu_D(W), \sigma_D^2(W))$ because X also follows a normal distribution. From Eq. (24), we can calculate the mean and variance of D_{approx} as functions of W :

$$\begin{aligned} \mu_D(W) &:= \mathbb{E}[D_{\text{approx}}] = Wc_1, \\ \sigma_D^2(W) &:= \text{Var}(D_{\text{approx}}) = W^4 c_2^2 \sigma_X^2, \end{aligned} \quad (25)$$

since $\mathbb{E}[X] = 0$ and $\text{Var}(X) = \sigma_X^2$. Differentiate the mean and variance w.r.t. W :

$$\frac{\partial \mu_D(W)}{\partial W} = c_1, \quad \frac{\partial \sigma_D^2(W)}{\partial W} = 4W^3 c_2^2 \sigma_X^2. \quad (26)$$

Invoking Lemma 1 with the parameter $\theta = W$, mean $\mu_D(W)$ and variance $\sigma_D^2(W)$, we obtain the Fisher information of the approximate normal model:

$$\begin{aligned} \mathcal{I}_D(W) &\approx \frac{1}{\sigma_D^2(W)} \left(\frac{\partial \mu_D(W)}{\partial W} \right)^2 + \frac{1}{2\sigma_D^4(W)} \left(\frac{\partial \sigma_D^2(W)}{\partial W} \right)^2 \\ &= \frac{c_1^2}{W^4 c_2^2 \sigma_X^2} + \frac{1}{2W^8 c_2^4 \sigma_X^4} \left(4W^3 c_2^2 \sigma_X^2 \right)^2 \\ &= \frac{c_1^2}{W^4 c_2^2 \sigma_X^2} + \frac{8}{W^2} = \frac{2}{W^2} \left(\frac{c_1^2}{2W^2 c_2^2 \sigma_X^2} + 4 \right). \end{aligned} \quad (27)$$

Therefore, we have the following conclusion: in this approximation,

$$\mathcal{I}_D(W) \geq \left(\frac{c_1^2}{2W^2 c_2^2 \sigma_X^2} + 4 \right) \cdot \mathcal{I}_Y(W). \quad (28)$$

This completes the proof of Theorem 1. $\square$

Example Queries in Query Set Q

Base Query:

Complete the following code: `def will_it_fly(q,w):` " " Write a function that returns True if the object q will fly, and False otherwise. The object ...

Perturbed Query:

1. Complete the following code: `def will_it_fly(q,w):` " " Write a function that returns **Fact** if the object q will **flies**, and False **not**. The object
2. Complete the following code: `def will_it_fly(q,w):` " " **Book** a function that returns **Fact** if the object q will fly, and False **simply**. The object

Figure 4: Example queries in the Query Set Q .

Algorithm 1 ZEROPRINT Query Set Construction

Input: D_{source} : Source dataset for base queries (e.g., HumanEval)

n : Number of base queries to select

m : Number of perturbed queries to generate per base query

r : Number of words to replace in each perturbation

k : Number of top semantically similar candidates to consider

E_{word} : Pre-trained word embedding model (e.g., GloVe)

Output: Q : The query set containing base and perturbed queries

```

1:  $Q_b \leftarrow \emptyset$  # Initialize set for base queries
2:  $Q \leftarrow \emptyset$  # Initialize the final query set
3: for  $i \leftarrow 1$  to  $n$  do
4:    $x_i \leftarrow \text{SampleAndFormat}(D_{\text{source}})$  # e.g., "Complete the code: [snippet]"
5:    $Q_b \leftarrow Q_b \cup \{x_i\}$ 
6: end for
7:  $Q \leftarrow Q \cup Q_b$  # Add all base queries to the final set
8: for each base query  $x_i \in Q_b$  do
9:   for  $j \leftarrow 1$  to  $m$  do
10:     $x'_{i,j} \leftarrow x_i$  # Create a copy of the base query
11:     $W_{\text{replaceable}} \leftarrow \text{SelectRandomWords}(x_i, r)$  # Select  $r$  words to replace
12:    for each word  $w \in W_{\text{replaceable}}$  do
13:       $C_{\text{candidates}} \leftarrow E_{\text{word}}.\text{TopKSimilar}(w, k)$  # Find top- $k$  similar words
14:       $w_{\text{replacement}} \leftarrow \text{RandomChoice}(C_{\text{candidates}})$ 
15:       $x'_{i,j} \leftarrow \text{ReplaceWord}(x'_{i,j}, w, w_{\text{replacement}})$ 
16:    end for
17:     $Q \leftarrow Q \cup \{x'_{i,j}\}$  # Add the perturbed query to the final set
18:  end for
19: end for
20: return  $Q$ 
```

B Implementation Details

Introduction to LEAFBENCH. LEAFBENCH is a benchmark that facilitates a unified evaluation of LLM fingerprinting methods [37]. It includes 7 base model lineages, including Qwen2.5-7B, Qwen2.5-14B, Llama3.1-8B, Mistral-7B-v0.3, Gemma-2-2B, TinyLlama-1.1B-v1.0, and Llama-2-7B. LEAFBENCH also implements 13 different post-development techniques that may be adopted to adjust the behavior of the LLMs. These techniques include: (1) pre-training, (2) instructional tuning, (3) fine-tuning, (4) parameter-efficient fine-tuning, (5) quantization, (6) model merging, (7) distillation, (8) general-purpose system prompts, (9) role-playing system prompts, (10) chain-of-thought prompts, (11) sampling strategies, (12) retrieval-augmented generation (RAG), (13) adversarial manipulation. LEAFBENCH includes a total number of 149 model instances.

Detailed Settings for ZEROPRINT. To generate the base queries, we sample code snippets from HumanEval, take the first 20 words, and then prepend them with "Complete the following code:" to form a query for a completion task. When conducting word substitutions, we use GloVe [30] as the word embedding model. For each base query, we randomly replace 3 words with one of the top-10 similar words. The example queries are shown in Figure 4 and the pseudocodes of query set construction and fingerprint generation in ZEROPRINT are in Algorithm 1&2. The coefficient α in Ridge

Algorithm 2 ZEROPRINT Fingerprint Generation

Input: $\mathcal{M}$: The target black-box LLM
 $Q_b = \{x_i\}_{i=1}^n$: Set of base queries
 $Q_p = \{x'_{i,j}\}_{j=1}^m \}_{i=1}^n$: Set of perturbed queries
 E : Pre-trained sentence embedding model
 t : Number of times to query the model for each input
 α : Regularization hyperparameter for Ridge Regression

Output: J_M : The final fingerprint (aggregated Jacobian matrix)

```

1:  $J_{list} \leftarrow []$  # Initialize list to store local Jacobians
2: for each base query  $x_i \in Q_b$  do
3:    $\Delta X_i, \Delta Y_i \leftarrow [], []$  # Initialize lists for input/output differences
4:   # Get stable embedding for the base query's output
5:    $\{y_{i,k}\}_{k=1}^t \leftarrow \text{QueryModelMultipleTimes}(\mathcal{M}, x_i, t)$ 
6:    $\bar{e}_{y_i} \leftarrow \frac{1}{t} \sum_{k=1}^t E(y_{i,k})$ 
7:    $e_{x_i} \leftarrow E(x_i)$ 
8:   for each perturbed query  $x'_{i,j}$  for  $x_i$  do
9:     # Get stable embedding for the perturbed query's output
10:     $\{y'_{i,j,k}\}_{k=1}^t \leftarrow \text{QueryModelMultipleTimes}(\mathcal{M}, x'_{i,j}, t)$ 
11:     $\bar{e}_{y'_{i,j}} \leftarrow \frac{1}{t} \sum_{k=1}^t E(y'_{i,j,k})$ 
12:     $e_{x'_{i,j}} \leftarrow E(x'_{i,j})$ 
13:    # Calculate input and output embedding differences
14:     $\Delta x_{i,j} \leftarrow e_{x'_{i,j}} - e_{x_i}$ 
15:     $\Delta y_{i,j} \leftarrow \bar{e}_{y'_{i,j}} - \bar{e}_{y_i}$ 
16:    Append  $\Delta x_{i,j}$  to  $\Delta X_i$ 
17:    Append  $\Delta y_{i,j}$  to  $\Delta Y_i$ 
18:   end for
19:   # Estimate the local Jacobian using Ridge Regression
20:    $J_i \leftarrow \arg \min_J \sum_{j=1}^m \|\Delta y_{i,j} - J \Delta x_{i,j}\|_2^2 + \alpha \|J\|_F^2$ 
21:   Append  $J_i$  to  $J_{list}$ 
22: end for
23:  $J_M \leftarrow \frac{1}{n} \sum_{i=1}^n J_{list}[i]$ 
24: return  $J_M$ 

```

Regression is set to 0.001. In our experiments, we utilize NVIDIA RTX 4090 GPUs with a total of 96GB VRAM.

Detailed Settings for the Baseline Fingerprinting Methods. In our experiments, we take 1 white-box method and 4 state-of-the-art black-box methods as our baseline. For fair comparison, following [54], we also set the query limit to $q = 200$ while applying these methods. The detailed settings are as follows.

- **REEF** [54]: REEF is a forward-pass white-box fingerprinting method that relies on the intermediate representations as fingerprints. We strictly follow the original setting and sample 200 queries from TruthfulQA [21] as the input queries.
- **LLMmap** [29]: LLMmap is an untargeted black-box fingerprinting method. We adopt the 8 carefully-crafted prompts as the input queries and utilize the pre-trained open-set feature extraction model provided in its open-source repository.
- **MET** [9]: MET is also a untargeted black-box fingerprinting method that uses text completion prompts as input queries. The original MET uses 25 samples. Therefore, we run MET 8 times, leading to a total of 200 queries.
- **SEF** [37]: SEF is an untargeted black-box fingerprinting method that straightforwardly uses the sentence embeddings of the outputs. Following original setting, we use Qwen3-Embedding-4B as the sentence embedding model and sample 200 queries from four different datasets, DROP [7], Ethics [14], PubMedQA [18], and HumanEval [3].
- **TRAP** [11]: TRAP is a targeted black-box fingerprinting method that leverages GCG [59] to generate targeted query-response pairs. For each query, we use GCG to optimize 100 iterations.

Table 5: The fingerprint similarities across different LLMs while auditing closed-source LLMs using ZEROPRINT.

Type	Derivative	Independent Models		
Models	GPT-4o	Gemini-2.5	Claude-4.5	DeepSeek-V3
GPT-4o	0.943	0.529	0.570	0.578

Detailed Settings of Ablation Studies. For the ablation study of different query data, except the HumanEval dataset [3], we select four other datasets as the source of the base queries, including the generative QA dataset TruthfulQA [21], the extractive QA dataset SQuAD [31], the complicated reasoning QA dataset DROP [7], and a text completion dataset Wikipedia [8]. For the Wikipedia dataset, similar to HumanEval, we sample a text snippet with 20 words and prepend a completion instruction (i.e., “Continue the following text:”) to construct a completion prompt. For the ablation study of different sentence embedding models, we select the following 5 models: all-mpnet-base-v2 (MPNet), all-MiniLM-L6-V2 (MiniLM) [32], potion-multilingual-128M (Model2Vec) [45], Qwen3-Embedding-0.6B (Qwen3) [56], and EmbeddingGemma-300m (EmbeddingGemma) [36]. They are all classic or state-of-the-art sentence embedding models with different embedding dimensions.

Detailed Settings of Adaptive Attacks. For input paraphrasing attacks, we utilize three small LLMs: Qwen2-0.5B-Instruct [41], OLMo-2-0425-1B-Instruct [27], and Gemma-2-2b-it [40], to rewrite the input queries. The experiments of adaptive attacks are conducted on Qwen2.5-7B-Instruct and Llama3.1-8B-Instruct models.

Detailed Settings of the Overhead Evaluation. The overhead evaluation in Section 5.5 is conducted on four derivative models from the Qwen2.5-7B lineage. For fair comparison, we utilize 1 NVIDIA RTX 4090 48GB GPU to compute the runtime of all the evaluated LLM fingerprinting methods.

C Case Study of Closed-source Audit

In this section, we include a case study of auditing closed-source LLMs using ZEROPRINT. Specifically, we utilize GPT-4o-mini (GPT-4o) as the source model and the same GPT-4o-mini API as the derivative, and take three other LLMs, Gemini-2.5-Flash-Lite (Gemini-2.5), Claude-Haiku-4.5 (Claude-4.5), and DeepSeek-V3, as independent models. The results in Table 5 show that the fingerprint similarity of fingerprints from the same GPT-4o-mini model is 0.943, which is close to 1, while other independent LLMs exhibit low similarity (≈ 0.50). These results demonstrate the feasibility and potential of our ZEROPRINT to audit closed-source, black-box LLMs.

D Ethic Statement

The unauthorized usage of LLM has raised significant concerns about copyright infringements, while LLM fingerprinting offers a promising solution. In this paper, we propose a novel black-box fingerprinting method, ZEROPRINT. Our method is designed for the purely defensive purpose of copyright auditing and does not introduce new threats or vulnerabilities. Furthermore, our work utilizes publicly available datasets, such as HumanEval, for query construction and does not infringe on the privacy of any individual. The research does not involve any human subjects. As such, this work does not raise any major ethical issues.